

Automatically Generate More Operators

Document Number: P1046R1

Date: 2019-09-26

Author: David Stone (david.stone@uber.com, david@doublewise.net)

Audience: Evolution Working Group (EWG), Library Evolution Working Group

Summary

This proposal follows the lead of operator<=> (see [Consistent Comparison](#)) by generating rewrite rules if a particular operator does not exist in current code. All of these operators would support `= default` to explicitly opt in and `= delete` to explicitly opt out.

- Rewrite `lhs += rhs` as `lhs = std::move(lhs) + rhs`, but evaluate `lhs` only once
- Rewrite `lhs -= rhs` as `lhs = std::move(lhs) - rhs`, but evaluate `lhs` only once
- Rewrite `lhs *= rhs` as `lhs = std::move(lhs) * rhs`, but evaluate `lhs` only once
- Rewrite `lhs /= rhs` as `lhs = std::move(lhs) / rhs`, but evaluate `lhs` only once
- Rewrite `lhs %= rhs` as `lhs = std::move(lhs) % rhs`, but evaluate `lhs` only once
- Rewrite `lhs &= rhs` as `lhs = std::move(lhs) & rhs`, but evaluate `lhs` only once
- Rewrite `lhs |= rhs` as `lhs = std::move(lhs) | rhs`, but evaluate `lhs` only once
- Rewrite `lhs ^= rhs` as `lhs = std::move(lhs) ^ rhs`, but evaluate `lhs` only once
- Rewrite `lhs <<= rhs` as `lhs = std::move(lhs) << rhs`, but evaluate `lhs` only once
- Rewrite `lhs >>= rhs` as `lhs = std::move(lhs) >> rhs`, but evaluate `lhs` only once
- Rewrite `++x` as `x += 1`
- Rewrite `--x` as `x -= 1`
- Rewrite `x++` as `++x` but return a copy of the original value
- Rewrite `x--` as `--x` but return a copy of the original value
- Rewrite `lhs->rhs` as `(*lhs).rhs`
- Rewrite `lhs->*rhs` as `(*lhs).*rhs`

Revision History

Changes in R1: Added operator->*. Added more detail on how postfix operators would work. Updated section on allocators in light of recent changes to the standard from P1165. Update some code examples to take advantage of implicit moves from rvalue references. Added standard library impact. Added discussion on other options for postfix operator++ and operator--.

Design Goals

There are certain well-defined patterns for how certain operators "should" be written. Many of these guidelines are widely followed and correct, leading to large amounts of code duplication. Some of these guidelines are widely followed and less optimal since C++11, leading to an even worse situation. Worst of all, some guidelines do not exist because the "correct" code cannot be written in C++. This proposal attempts reduce boilerplate, improve performance, increase regularity, and remove bugs.

One of the primary goals of this paper is that users should have very little reason to write their own version of an operator that this paper proposes generating. It would be a strong indictment if there are many examples of well-written code for which this paper provides no simplification, so a fair amount of time in this paper will be spent looking into the edge cases and making sure that we generate the right thing. At the very least, types that would not have the correct operator generated should not generate an operator at all. In other words, it should be uncommon for users to define their own versions (`= default` should be good enough most of the time), and it should be very rare that users want to suppress the generated version (`= delete` should almost never appear).

Arithmetic, Bitwise, and Compound Assignment Operators

For all binary arithmetic and bitwise operators, there are corresponding compound assignment operators. There are two obvious ways to implement these pairs of operators without code duplication: implement `a += b` in terms of `a = a + b` or implement `a = a + b` in terms of `a += b`. Using the compound assignment operator as the basis has a tradition going back to C++98, so let's consider what that code looks like using `string` as our example:

```
string & operator+=(string & lhs, string const & rhs) {
    lhs.append(rhs);
    return lhs;
}

string & operator+=(string & lhs, string && rhs) {
    if (lhs.size() + rhs.size() <= lhs.capacity()) {
        lhs.append(rhs);
        return lhs;
    } else {
        rhs.insert(0, lhs);
        lhs = std::move(rhs);
        return lhs;
    }
}

string operator+(string const & lhs, string const & rhs) {
    string result;
    result.reserve(lhs.size() + rhs.size());
```

```

        result = lhs;
        result.append(rhs);
        return result;
    }
    string operator+(string const & lhs, string && rhs) {
        rhs.insert(0, lhs);
        return rhs;
    }
    template<typename String> requires std::same_as<std::decay_t<String>,
    string operator+(string && lhs, String && rhs) {
        lhs += std::forward<String>(rhs);
        return lhs;
    }
}

```

This does not seem to save us as much as we might like. We can have a single template definition of `string + string` when we take an rvalue for the left-hand side argument, but when we have an lvalue as our left-hand side we need to write two more overloads (neither of which really benefits from `operator+=`). It may seem tempting at first to take this approach due to historical advice, but taking advantage of move semantics eliminates most code savings. This does not feel like a generic answer to defining one class of operators.

What happens if we try to use `operator+` as the basis for our operations? The initial reason that C++03 code typically defined its operators in terms of `operator+=` was for efficiency, but with the addition of move semantics to C++11, we can regain this efficiency. What does the ideal `string + string` implementation look like?

```

// Same as in the previous example
string operator+(string const & lhs, string const & rhs) {
    string result;
    result.reserve(lhs.size() + rhs.size());
    result = lhs;
    result.append(rhs);
    return result;
}
// Same as in the previous example
string operator+(string const & lhs, string && rhs) {
    rhs.insert(0, lhs);
    return rhs;
}
// Very similar to the equivalent in the previous example
string operator+(string && lhs, string const & rhs) {
    lhs.append(rhs);
}

```

```

        return lhs;
    }
    // Compare to previous example's operator+=(string &, string &&)
    string operator+(string && lhs, string && rhs) {
        return (lhs.size() + rhs.size() <= lhs.capacity()) ?
            std::move(lhs) + std::as_const(rhs) :
            std::as_const(lhs) + std::move(rhs);
    }

    template<typename String> requires std::same_as<std::decay_t<String>,
    string & operator+=(string & lhs, String && rhs) {
        lhs = std::move(lhs) + std::forward<String>(rhs);
        return lhs;
    }

```

This mirrors the implementation if `operator+=` is the basis operation: we still end up with five functions in our interface. One important difference between the two is that in the first case, where `operator+=` is the basis operation, we needed to define `operator+=` and `operator+`. Defining only `operator+=` and trying to generically define `operator+` in terms of that gives us an inefficiently generated `operator+`. In the second case, however, we need to define only `operator+` specially, and our one `operator+=` overload will be maximally efficient.

One interesting thing of note here is that the `operator+=` implementation does not know anything about its types other than that they are addable and assignable. Compare that with the `operator+` implementations from the first piece of code where they need to know about sizes, reserving, and insertion because `operator+=` alone is not powerful enough. There are several more counterexamples that argue against `operator+=` being our basis operation, but most come down to the same fact: `operator+` does not treat either argument -- lhs or rhs -- specially, but `operator+=` does.

The first counterexample is `std::string` (again). We can perform `string += string_view`, but not `string_view += string`. If we try to define `operator+` in terms of `operator+=`, that would imply that we would be able to write `string + string_view`, but not `string_view + string`, which seems like an odd restriction. The class designer would have to manually write out both `+=` and `+`, defeating our entire purpose.

The second counterexample is the `bounded::integer` library and `std::chrono::duration`. `bounded::integer` allows users to specify compile-time bounds on an integer type, and the result of arithmetic operations reflects the new bounds based on that operation. For instance, `bounded::integer<0, 10> + bounded::integer<1, 9> => bounded::integer<1, 19>`. `duration<LR, Period> + duration<RR, Period> => duration<common_type_t<LR, RR>, Period>` If `operator+=` is the basis, it requires that

the return type of the operation is the same as the type of the left-hand side of that operation. This is fundamentally another instance of the same problem with `string_view + string`.

The third counterexample is all types that are not assignable (for instance, if they contain a `const` data member). This restriction makes sense conceptually. An implementation of `operator+` requires only that the type is addable in some way. Thanks to guaranteed copy elision, we can implement it on most types in such a way that we do not even require a move constructor. An implementation of `operator+=` requires that the type is both addable and assignable. We should make the operations with the broadest applicability our basis.

Let's look a little closer at our `operator+=` implementation that is based on `operator+` and see if we can lift the concepts used to make the operation more generic. For strings, we wrote this:

```
template<typename String> requires std::same_as<std::decay_t<String>,
string & operator+=(string & lhs, String && rhs) {
    lhs = std::move(lhs) + std::forward<String>(rhs);
    return lhs;
}
```

The first obvious improvement is that the types should not be restricted to be the same type. Users expect that if `string = string + string_view` compiles, so does `string += string_view`. We are also not using anything string specific in this implementation, and our goal is to come up with a single `operator+=` that works for all types. With this, we end up with:

```
template<typename LHS, typename RHS>
LHS & operator+=(LHS & lhs, RHS && rhs) {
    lhs = std::move(lhs) + std::forward<RHS>(rhs);
    return lhs;
}
```

Are there any improvements we can make to this signature? One possible problem is that we unconditionally return a reference to the lhs, regardless of the behavior of the assignment operator. This is what is commonly done, but there are some use cases for not doing so. The most compelling example might be `std::atomic`, which returns `T` instead of `atomic<T>` in `atomic<T> = T`, and this behavior is reflected in the current compound assignment operators. An improvement to our previous signature, therefore, is the following implementation:

```
template<typename LHS, typename RHS>
decltype(auto) operator+=(LHS & lhs, RHS && rhs) {
    return lhs = std::move(lhs) + std::forward<RHS>(rhs);
}
```

Now our compound assignment operator returns whatever the assignment operator returns. We have a technical reason to make this change, but does it match up with our expectations? Conceptually, `operator+=` should perform an addition followed by an assignment, so it seems reasonable to return whatever assignment returns, and this mirrors the (only?) type in the standard library that returns something other than `*this` from its assignment operator. This is also the behavior that would naturally fall out of using a rewrite rule such that the expression `lhs += rhs` is rewritten to `lhs = std::move(lhs) + rhs`, which is what is proposed by this paper.

One final question might be about that somewhat strange looking `std::move(lhs)` piece. You might wonder whether this is correct or safe, since it appears to lead to a self-move-assignment. Fortunately, we sidestep this problem entirely because there is actually no self-move-assignment occurring here. For well-behaved types, `operator+` might accept its argument by rvalue-reference, but that function then returns by value. This means that the object assigned to `lhs` is a completely separate object, and there are no aliasing concerns.

Single evaluation

Because this proposal works in terms of expression rewrites, there is a pitfall in simply rewriting `lhs @= rhs` as `lhs = std::move(lhs) @ rhs`: the expression `lhs` appears twice. We want essentially the same semantics that a function would give us: exactly one evaluation of `lhs`. In other words, the specification would have to be equivalent to something like

```
[&] {  
    auto && __temp = lhs;  
    return static_cast<decltype(__temp)>(__temp) = std::move(__temp) @  
}()
```

Allocators

There is one major remaining question here: how does this interact with allocators? In particular, we might care about the values in `std::allocator_traits` for `propagate_on_container_copy_assignment`, `propagate_on_container_move_assignment`, `propagate_on_container_swap` and `select_on_container_copy_construction`. The move constructor of a standard container has no special interaction with allocators -- in particular, for all containers (except `std::array`) the move constructor is constant time, regardless of the type or value of the source allocator.

With an eye toward allocators, let's look again at the case of `string` and the operator overloads (with minor modifications to our example code to match the current specification of `operator+` with regards to allocators):

```
string operator+(string const & lhs, string const & rhs) {  
    string result(allocator_traits::select_on_container_copy_construct
```

```

        result.reserve(lhs.size() + rhs.size());
        result.append(lhs);
        result.append(rhs);
        return result;
    }
    string operator+(string && lhs, string const & rhs) {
        lhs.append(rhs);
        return lhs;
    }
    string operator+(string const & lhs, string && rhs) {
        rhs.insert(0, lhs);
        return rhs;
    }
    string operator+(string && lhs, string && rhs) {
        return (lhs.size() + rhs.size() <= lhs.capacity() or lhs.get_allocator() == rhs.get_allocator()) ?
            std::move(lhs) + std::as_const(rhs) :
            std::as_const(lhs) + std::move(rhs);
    }

    template<typename LHS, typename RHS>
    decltype(auto) operator+=(LHS & lhs, RHS && rhs) {
        return lhs = std::move(lhs) + std::forward<RHS>(rhs);
    }

```

Our goal here is to make sure that given the proper definitions of `operator+`, our generic definition of `operator+=` will still do the correct thing. This is somewhat tricky, so we'll walk through this step by step. First, let's consider the case where `rhs` is `string const &`. We end up with this instantiation:

```

    string & operator+=(string & lhs, string const & rhs) {
        return lhs = std::move(lhs) + rhs;
    }

```

That will call this overload of `operator+`:

```

    string operator+(string && lhs, string const & rhs) {
        lhs.append(rhs);
        return lhs;
    }

```

Before calling this function, nothing has changed with allocators because all we have done is bound variables to references. Inside the function, we make use of the allocator associated with `lhs`, if necessary, which is exactly what we want (the final object will be `lhs`). We then return `lhs`, which move constructs the return value. Move construction is guaranteed to move in the allocator, so our resulting value has the same allocator as `lhs` did at the start of the function. Going back up the stack, we then move assign this constructed string back into `lhs`. This will be fast because those allocators compare equal. This is true regardless of the value of `propagate_on_container_move_assignment`, since propagation is unnecessary due to the equality.

The other case to consider is when `rhs` is `string &&`. Let's walk through the same exercise:

```
string & operator+=(string & lhs, string && rhs) {  
    return lhs = std::move(lhs) + std::move(rhs);  
}
```

That will call this overload of `operator+`:

```
string operator+(string && lhs, string && rhs) {  
    return (lhs.size() + rhs.size() <= lhs.capacity() or lhs.get_allocator().  
        == rhs.get_allocator()) ?  
        std::move(lhs) + std::move(rhs) :  
        std::move(lhs) + rhs;  
}
```

If `lhs` has enough capacity for the resulting object, we forward to the same overload we just established as being correct for our purposes. If the allocators are different, it means we care which allocator we use, and so we should also fall back to that same overload. In that second branch, we end up returning `rhs` with its allocator, but this is acceptable because we know the allocators are equal.

In other words, this proposal properly handles the allocator model of `std::string` and would presumably properly handle a hypothetical `big_integer` type.

Note that these examples still hold if we add in overloads for `string_view` and `char const *`. For simplicity, they have been left out.

Does this break the stream operators?

For "bitset" types, the expressions `lhs << rhs` and `lhs >> rhs` mean something very different than for other types, thanks to those operators being used for stream insertion / extraction.

Fortunately, this proposal is unlikely to lead to the creation of a strange `<<=` or `>>=` operator with stream arguments. The existing stream operators require the stream to start out on the left-hand

side. This means that the only compound assignment operators that this proposal attempts to synthesize are `stream <<= x` and `stream >>= x`. For this to compile using `std::istream` or `std::ostream`, the user must have overloaded `operator>>` or `operator<<` to return something other than the standard of `stream &` and instead return something that can be used as the source of an assignment to a stream object. There are not many types that meet this criterion, so the odds of this happening accidentally are slim.

Optimization opportunities

To ensure that we are not generating code that is difficult to optimize, it can be helpful to consider a simple `vector3d` class:

```
struct vector3d {  
    double x;  
    double y;  
    double z;  
};
```

This example uses a struct with three data members, but the generated code is identical for a struct containing a single array data member.

To see how some major compilers handle various ways of defining operator overloads for this type, you can see [this Godbolt link](#). The compiler flags used are maximum optimization with no special architecture-specific flags like `-march` or `-mtune`, but a cursory examination of various values for those flags showed it did not change interpretation of the results. For example:

- `llvm` generates identical instructions (in a slightly different order) for `+` defined directly, for `+=` defined in terms of `+`, and for `+=` defined directly, but it generates worse code for `+` defined in terms of `+=`. In other words, it is best to define `+=` in terms of `+`.
- `gcc` generates identical instructions (in a slightly different order) for all combinations. In other words, it does not matter which is the basis.
- `MSVC` generates identical instructions (in a slightly different order) for `+` defined directly and for `+=` defined directly, but it generates the same worse code for whichever operator is defined in terms of the other.
- `ICC` generates identical code for `+=` defined directly or in terms of `+`. It generates worse code for `+` defined in terms of `+=` than for `+` defined directly.

To summarize, it is better for `llvm` and `ICC` to define `+` as the basis operation, irrelevant for `gcc`, and on `MSVC` you want to define neither in terms of the other.

Recommendation

We should synthesize compound assignment operators following the same rules as we follow for `operator<=>`: the overload set for the compound assignment operators include a synthesized

operator, which is considered a worse match than a non-synthesized operator. In other words, `lhs @ rhs` has a candidate overload of `lhs = std::move(lhs) @ rhs`. Users can explicitly opt in to using such an operator rewrite by specifying `= default`. This is entirely for symmetry, as it is not especially useful, just like `bool operator!=(T, T) = default;` is not especially useful. They can opt out by using `= delete`. This section of the proposal would synthesize rewrite rules for the following operators:

- `operator+=`
- `operator-=`
- `operator*=`
- `operator/=`
- `operator%=`
- `operator&=`
- `operator|=`
- `operator^=`
- `operator<<=`
- `operator>>=`

Prefix Increment and Decrement

Recommendation

Rewrite `++a` as `a += 1`, and rewrite `--a` as `a -= 1`.

Postfix Increment and Decrement

Recommendation

Rewrite `a++` as something like `[&]{ auto __temp = a; ++a; return __temp; }()` with guaranteed NRVO (in other words, there is a single copy performed but no moves) and rewrite `a--` as something like `[&]{ auto __temp = a; --a; return __temp; }()` with guaranteed NRVO. This will work for most normal types, but fails to compile for `std::atomic` because `std::atomic` is non-copyable (the full list is in the library impact section). This is fine, because `std::atomic` would need to provide different functionality by returning a copy of the original `T`, rather than a copy of the original `std::atomic<T>`, so failing to compile is the correct outcome here.

There are several iterator adaptors added in C++20 that have a postfix `operator++` that returns `void` if the iterator they are adapting is not copyable (the `input_iterator` concept no longer requires copyability), and otherwise behave the same as prefix `operator++`. Is this behavior we would want to standardize at the language level? If so, this would simplify a lot of library specification, and if we believe this is an appropriate pattern for users to follow as well it seems to be the right thing to do. It is generally expected that prefix and postfix increment / decrement have

the same effects on the object and differ only in their return type, and it is generally acceptable to change a return type from `void` to something else, so this would be unlikely to silently add a behavior that would need to be changed by the library author in the future. If we don't think this is a good solution at the language level, maybe we should reconsider that behavior in the library.

Member of pointer (`->`)

This area has been well-studied for library solutions. This paper, however, traffics in rewrite rules (following the lead of `operator<=>`), not in terms of function calls. Because of this, we have one more option that the library-only solutions lack: we could define `lhs->rhs` as being equivalent to `(*lhs).rhs`. This neatly sidesteps all of the issues of library-only solutions (how do we get the address of the object? how do we handle temporaries?). It even plays nicely with existing rules around lifetime extension of temporaries. This solves many long-standing issues around proxy iterators that return by value.

Recommendation

Rewrite `lhs->rhs` as `(*lhs).rhs`.

Pointer to member of pointer (`->*`)

Same reasoning as `operator->`.

Recommendation

Rewrite `lhs->*rhs` as `(*lhs).*rhs`.

Library impact

Given the path we took for `operator<=>` of removing manual definitions of operators that can be synthesized and assuming we want to continue with that path by approving ["Do not promise support for function syntax of operators"](#), this will allow removing a large amount of specification in the standard library by replacing it with blanket wording that these rewrites apply. I have surveyed the standard library to get an overview of what would change in response to this, and to ensure that the changes would work properly.

`operator@=`

Types that can have all of their `operator@=` defaulted

- `basic_string`: Can remove all `operator+=` except the overload taking `initializer_list` (or we could add `basic_string + initializer_list`)
- `basic_string::iterator`: `+=`, `-=`

- `array::iterator`: `+=`, `-=`
- `deque::iterator`: `+=`, `-=`
- `vector::iterator`: `+=`, `-=`
- `valarray::iterator`: `+=`, `-=`
- `iota_view::iterator`: `+=`, `-=`
- `elements_view::iterator`: `+=`, `-=`
- `complex`: `+=`, `-=`, `*=`, `/=`
- `valarray`: `+=`, `-=`, `*=`, `/=`, `%=`, `^=`, `&=`, `|=`, `<<=`, `>>=`
- `chrono::day`: `+=`, `-=`
- `chrono::month`: `+=`, `-=`
- `chrono::year`: `+=`, `-=`
- `chrono::weekday`: `+=`, `-=`
- `chrono::year_month`: `+=`, `-=`
- `chrono::year_month_day`: `+=`, `-=`
- `chrono::year_month_day_last`: `+=`, `-=`
- `chrono::year_month_weekday`: `+=`, `-=`
- `chrono::year_month_weekday_last`: `+=`, `-=`
- "bitmask type": `^=`, `&=`, `|=` can be removed from the exposition
- `byte`: `^=`, `&=`, `|=`, `<<=`, `>>=`
- `bitset`: `^=`, `&=`, `|=`, `<<=`, `>>=`

Types that wrap another type and forward to its `+`, `+=`, `-`, and `-=`

- `reverse_iterator`
- `move_iterator`
- `counted_iterator`
- `transform_view::iterator`
- `chrono::duration` (also `*=`, `/=`, `%=`)
- `chrono::time_point`

All of these types have an `operator+` that calls some underlying type's `operator+` and an `operator+=` that call some underlying type's `operator+=`. We will have to decide whether we want to here. It is important to note that the new `random_access_iterator` concept and the Cpp17RandomAccessIterator requirements table already state that if your type is a random access iterator, the two operations must be equivalent (for some definition of equivalence). Also note that despite `elements_view::iterator` wrapping a user-defined iterator, it always calls `+=`.

For `chrono::duration`, the standard states that its template parameter "Rep shall be an arithmetic type or a class emulating an arithmetic type". It is unclear how true this emulation must be, but presumably it requires that arithmetic operations have the usual equivalences, in which case it should not matter which specific operators are called.

Types that mysteriously do not have `a + b` but do have `a += b`, even though they have `a / b` and `a /= b`

- `filesystem::path`

If we define `operator+` for this type, we could then get rid of `operator+=`. Pros: increased uniformity and better support for functional programming styles. Cons: Would make the following code valid:

```
std::list<char> range{0, 1, 2, 3, 4, 5};
std::filesystem::path p = "101";
std::regex(range.begin() + p);
// Note: This code is already valid if you replace `+` with `/`
```

The synthesized version of `operator/=` is correct for `filesystem::path`.

Correctly unaffected types

- `slice_array`
- `gslice_array`
- `mask_array`
- `indirect_array`
- `atomic_ref`
- `atomic`

These have only compound assignment operators.

Prefix `operator++` and `operator--`

Types that have the operator now and it behaves the same as the synthesized operator (`a += 1`)

- `basic_string::iterator`
- `array::iterator`
- `deque::iterator`
- `vector::iterator`
- `valarray::iterator`
- `atomic<integral>` and `atomic<pointer>`
- `atomic_ref<integral>` and `atomic_ref<pointer>`

Types that have the operator now but do not or might not have `+=` or `-=` (no change from this proposal)

- `forward_list::iterator` (`++` only)
- `list::iterator`
- `map::iterator`
- `set::iterator`
- `multimap::iterator`

- `multiset::iterator`
- `unordered_map::iterator`
- `unordered_set::iterator`
- `unordered_multimap::iterator`
- `unordered_multiset::iterator`
- `back_insert_iterator` (++) only)
- `front_insert_iterator` (++) only)
- `insert_iterator` (++) only)
- `reverse_iterator`
- `move_iterator`
- `common_iterator`
- `counted_iterator`
- `istream_iterator` (++) only)
- `ostream_iterator` (++) only)
- `istreambuf_iterator` (++) only)
- `ostreambuf_iterator` (++) only)
- `iota_view::iterator`
- `filter_view::iterator`
- `transform_view::iterator`
- `join_view::iterator` (++) only)
- `split_view::outer_iterator` (++) only)
- `split_view::inner_iterator` (++) only)
- `basic_istream_view::iterator` (++) only)
- `elements_view::iterator`
- `directory_iterator` (++) only)
- `recursive_directory_iterator` (++) only)
- `regex_iterator` (++) only)
- `regex_token_iterator` (++) only)

Needs to keep existing version because the rewrite would not compile

- `chrono::duration`
- `chrono::time_point`
- `chrono::day`
- `chrono::month`
- `chrono::year`
- `chrono::weekday`

Does not have operator++ or operator-- now, and should not have it (= delete)

- `basic_string` (operator++ only)
- `valarray`
- `slice_array`
- `gslice_array`

- `mask_array`
- `indirect_array`
- `filesystem::path` (operator++ only)

Does not have operator++ or operator-- and unclear whether it should have it

- `complex`
- `atomic<floating_point>`

Postfix operator++ and operator--

Types that have the operator now and it behaves the same as the synthesized operator even if postfix does not work for non-copyable types

- `basic_string::iterator`
- `array::iterator`
- `deque::iterator`
- `vector::iterator`
- `forward_list::iterator` (++ only)
- `list::iterator`
- `map::iterator`
- `set::iterator`
- `multimap::iterator`
- `multiset::iterator`
- `unordered_map::iterator`
- `unordered_set::iterator`
- `unordered_multimap::iterator`
- `unordered_multiset::iterator`
- `valarray::iterator`
- `reverse_iterator`
- `back_insert_iterator` (++ only)
- `front_insert_iterator` (++ only)
- `insert_iterator` (++ only)
- `istream_iterator` (++ only)
- `chrono::duration`
- `chrono::time_point`
- `chrono::day`
- `chrono::month`
- `chrono::year`
- `chrono::weekday`
- `regex_iterator` (++ only)
- `regex_token_iterator` (++ only)

Types that have the operator now and it behaves the same as the synthesized operator only if postfix returns void for non-copyable types

- `move_iterator`
- `iota_view::iterator`
- `filter_view::iterator`
- `transform_view::iterator`
- `join_view::iterator`
- `split_view::outer_iterator`
- `split_view::inner_iterator`
- `basic_istream_view::iterator`
- `elements_view::iterator`

`basic_istream_view` always returns `void` from its postfix operator`++`. The others wrap another iterator type and always call the prefix operator`++` or operator`--` (where applicable) on the wrapped type. If the type they are wrapping does not meet `forward_iterator` (it is an `input_iterator` only), then they return `void` from their postfix operator`++` (`input_iterator` does not support operator`--` so it does not get special treatment), otherwise they return `*this` by reference.

Types that defer to a wrapped postfix `++` for iterators that do not meet `forward_iterator`

- `common_iterator`
- `counted_iterator`

Regardless of what happens with this proposal, these types sometimes return their own type and call the wrapped type's prefix operator`++`, and returns the result of calling the wrapped type's postfix operator`++`. This is an inconsistent design that probably deserves further discussion.

Types that return a reference from postfix `++`

- `ostream_iterator`
- `ostreambuf_iterator`

Types that would do something bad with the synthesized version and thus need to keep their existing overload

- `istreambuf_iterator` has a postfix operator`++` that returns a proxy that keeps the old value alive
- `directory_iterator` has no postfix, but it is a copyable `input_iterator`.
- `recursive_directory_iterator` has no postfix, but it is a copyable `input_iterator`

Types that would return less information than is currently returned and thus need to keep their existing overload

- `atomic_ref`
- `atomic`

These overloads would need to stay. The postfix operators return a copy of the underlying value as it was before the increment. Under the language rules of this proposal, if the manual postfix

operators were removed from `std::atomic`, the postfix operator would either not exist (because atomics are non-copyable) or they would return `void` instead (depending on how we resolve this issue). If the manual postfix operators were removed from `atomic_ref`, it would return a copy of the `atomic_ref` rather than the value.

operator->

Types that will gain operator-> and this is a good thing

- `move_iterator` currently has a deprecated `operator->`
- `counted_iterator`
- `istreambuf_iterator`
- `istreambuf_iterator::proxy` (exposition only type)
- `iota_view::iterator`
- `transform_view::iterator`
- `split_view::outer_iterator`
- `split_view::inner_iterator`
- `basic_istream_view::iterator`
- `elements_view::iterator`

Most of these are iterators that return either by value or by `decltype(auto)` from some user-defined function. It is not possible to safely and consistently define `operator->` for these types, so we do not always do so, but under this proposal they would all do the right thing.

Types that will technically gain operator-> but it is not observable

- `insert_iterator`
- `back_insert_iterator`
- `front_insert_iterator`
- `ostream_iterator`

The insert iterators and `ostream_iterator` technically gain an `operator->`, but `operator*` returns a reference to `*this` and the only members of those types are types, constructors, and operators, none of which are accessible through `operator->` using the syntaxes that are supported to access the standard library.

Types that will gain operator-> and it is a bad thing

- `ostreambuf_iterator`

`ostreambuf_iterator` is the one bad example for which we would want to explicitly delete `operator->`. It has an `operator*` that returns `*this`, and it has a member function `failed()`, so it would allow calling `it->failed()` with the same meaning as `it.failed()`.

Types that have operator-> now and it behaves the same as the synthesized operator

All types in this section have an `operator->` that is identical to the synthesized version if we do not wish to support users calling with the syntax `thing.operator->()`.

- `optional`
- `unique_ptr` (single object)
- `shared_ptr`
- `weak_ptr`
- `basic_string::iterator`
- `basic_string_view::iterator`
- `array::iterator`
- `deque::iterator`
- `forward_list::iterator`
- `list::iterator`
- `vector::iterator`
- `map::iterator`
- `multimap::iterator`
- `set::iterator`
- `multiset::iterator`
- `unordered_map::iterator`
- `unordered_set::iterator`
- `unordered_multimap::iterator`
- `unordered_multiset::iterator`
- `span::iterator`
- `istream_iterator`
- `valarray::iterator`
- `tzdb_list::const_iterator`
- `filesystem::path::iterator`
- `directory_iterator`
- `recursive_directory_iterator`
- `match_results::iterator`
- `regex_iterator`
- `regex_token_iterator`

Types that defer to a wrapped operator-> if it exists

- `reverse_iterator`
- `common_iterator`
- `filter_view::iterator` (effectively, it returns the iterator itself from `operator->`)
- `join_view::iterator` (effectively, it returns the iterator itself from `operator->`)

All of these types currently define their `operator->` as deferring to the base iterator's `operator->`. However, the Cpp17InputIterator requirements specify that `a->m` is exactly equivalent to `(*a).m`, so anything a user passes to `reverse_iterator` must already meet this. The other three iterators are new in C++20 and require `input_or_output_iterator` of their parameter, which says nothing about `->`. This means that based on what we have promised about

our interfaces, we could implement all of these under the language proposal if we change `common_iterator`, `filter_view::iterator`, and `join_view::iterator` for C++20.

iterator_traits

`std::iterator_traits<I>::pointer` is essentially defined as `typename I::pointer` if such a type exists, otherwise `decltype(a.operator->())` (where `a` is some value of type `I`) if that expression is well-formed, otherwise `void`. The type appears to be unspecified for iterators into any standard container, depending on how you read the requirements. The only relevant requirement on standard container iterators (anything that meets `Cpp17InputIterator`) are that `a->m` is equivalent to `(*a).m`. We never specify that any other form is supported, nor do we specify that any of them contain the member type `pointer`. There are three options here:

1. Change nothing. This would make `pointer` defined as `void` for types that have a synthesized `operator->`
2. Specify a further fallback of `decltype(std::addressof(*a))` to maintain current behavior and allow users to delete their own `operator->` without changing the results of `iterator_traits`
3. Deprecate or remove the `pointer` typedef, as it is not used anywhere in the standard except to define other `pointer` typedefs and it seems to have very little usefulness outside the standard.

My recommendation is either 2 or 3.

to_address and pointer_traits

[`pointer.conversion`] specifies `to_address` in terms of calling `p.operator->()`, so some thought will need to be put in there on what to do.

The following standard types can be used to instantiate `pointer_traits`:

- `T *`
- `unique_ptr`
- `shared_ptr`
- `weak_ptr`
- `span`

However, none of them are specified to have member `to_address`.

`std::to_address` is specified in terms of calling `operator->` with member function syntax. It uses this as a fallback if `pointer_traits<Ptr>::to_address(p)` is not well-formed. This leaves us with several options.

1. Leave this function as-is and specify that all of the types that currently have `operator->` have a specialization of `pointer_traits` that defines `pointer_traits<T>::to_address`
2. Specify that all types that currently have `operator->` work with `std::to_address`

3. Define a second fallback if `p.operator->()` is not valid that would be defined as `std::addressof(*p)`. This is similar to the question for `std::iterator_traits::pointer`.
4. Decide that iterators that do not satisfy `contiguous_iterator` are not sufficiently "pointer like", and thus should not be used with `std::to_address`. This would require changing the definition for C++20. We would then say that calling `std::to_address` works for all `contiguous_iterator` types. This just pushes the question down the road a bit (we still have to decide how that works and how users opt in to this), but it dramatically reduces the number of types that would have to do this (in the standard, it would just be 6 types: `basic_string::iterator`, `basic_string_view::iterator`, `array::iterator`, `vector<non_bool>::iterator`, `span::iterator`, and `valarray::iterator`). If we go this route, we need to decide on option 1, 2, or 3 for those iterator types.

1 and 2 feel like the wrong approach -- they would mean that authors of iterator types still need to define their own `operator->`, or they must specialize some class template (if we agree that the current semantics with regard to iterators are correct), or they must overload `to_address` and we make that a customization point found by ADL.

Summary of open questions

Other than the first question (which is both language and library), all of these questions are library questions.

- Should we synthesize a postfix `operator++` and `operator--` that return `void` for types that have the prefix version but are non-copyable? If not, should we remove such overloads from C++20 for the iterator adapter types mentioned in that section of this paper?
- For iterator adaptors, `chrono::duration`, and `chrono::time_point`, they currently have their `operator@=` defer to some user-defined type's `operator@=`, but we appear to have text requiring that to be equivalent to `operator@`. Do we want to maintain this wrapping behavior for all of those types, none of those types, or make a change to C++20 for `transform_view::iterator` and `counted_iterator` to make them not wrap?
- Should `complex` and `atomic<floating_point>` have `operator++` and `operator--`? They both support `a += 1` and their underlying type supports `++b`.
- Should `ostream_iterator` and `ostreambuf_iterator` be changed to return by value from postfix `operator++`?
- Should `reverse_iterator`, `common_iterator`, `filter_view::iterator`, and `join_view::iterator` continue to call an underlying iterator type's `operator->` in their `operator->` if it exists? If not, should we remove such functionality from C++20 for `common_iterator`, `filter_view::iterator`, and `join_view::iterator` and define them as returning `addressof(**this)`? This would technically not be a breaking change because we never promised for `reverse_iterator` that the implementation isn't already doing what the language proposal suggests.

- How should `iterator_traits<I>::pointer` be defined? a) Do nothing => get a void typedef for types that use the language feature. b) Specify a further fallback => `decltype(std::addressof(*a))`. c) Deprecate the typedef.
- How should `std::to_address` be defined?

Things with no recommendations to change

The following sections are operators that were evaluated, but for which there is not a recommendation to change.

Subscript (`operator[]`)

The subscript operator is a bit trickier. The problem here is that there are actually at least three different subscript operators with the same syntax. The first subscript operator is the subscript operator of iterators: `it[n]` is equivalent to `*(it + n)`. The second subscript operator is the subscript operator of random-access ranges: `range[n]` is equivalent to `begin(range)[n]`. The third subscript operator is lookup into an associative container: `dictionary[key]` is equivalent to `*dictionary.emplace(key).first`. This gets even more complicated when you consider that a type can model both a random-access range and an associative container (for instance: `flat_map`).

`operator-`

`a - b` is theoretically equivalent to `a + -b`. This is true in C++ only for types that accurately model their mathematical equivalents or else maintain a modular equivalence (for instance, unsigned types). The problem is that the expression `-b` might overflow. Even if that negation is well defined, there still may be overflow; consider `ptr - 1U`: if we rewrote that to be `ptr + -1U`, the pointer arithmetic would overflow. Attempting this rewrite is a dangerous non-starter.

How about the other way around? Perhaps we could define `-a` as `0 - a`? This could be a promising avenue, but it is not proposed in this paper. That exact definition would need some tweaking, as this would give pointers a unary minus operator with undefined behavior for non-null pointers (`0` is also a null pointer constant, allowing the conversion, and pointer arithmetic is undefined for out-of-bounds arithmetic, but this is a solvable problem).

References

- [Ranges TS: Post-Increment on Input and Output Iterators](#)
- [Movability of Single-pass Iterators](#)